

Computer Architectures

Exam of 12.02.2024 – part II

Iteration 4

*	n	*	*	*	*	*	*	*
*	n	W	W	*		*		*
*	n	*	*	*	*	*		*
*	n	*		*				*
*	n	*		*	*	*	S	*
*				*		e	S	*
*	*	*	*	*		*	S	*
*						*	S	*
*	*	*	*	*	*	*	S	*

Iteration 5

*	n	*	*	*	*	*	*	*
*	n	W	W	*		*		*
*	n	*	*	*	*	*		*
*	n	*		*			S	*
*	n	*		*	*	*	S	*
*	n			*	e	e	S	*
*	*	*	*	*		*	S	*
*						*	S	*
*	*	*	*	*	*	*	S	*

...

Iteration 11

*	n	*	*	*	*	*	*	*
*	n	W	W	*		*	S	*
*	n	*	*	*	*	*	S	*
*	n	*	S	*	e	e	S	*
*	n	*	S	*	*	*	S	*
*	n	W	W	*	e	e	S	*
*	*	*	*	*	n	*	S	*
*	e	e	e	e	n	*	S	*
*	*	*	*	*	*	*	S	*

The procedure returns the length of the longest path, i.e., the number of iterations executed. In the example, the return value is 11.

Computer Architectures

Exam of 12.02.2024 – part II

Question 2

We want to generate a maze randomly and then call the subroutine developed in the previous exercise to find the shortest path towards the exits.

A `NUM_ROWS * NUM_COLUMNS` matrix of chars is declared in C. `NUM_ROWS` and `NUM_COLUMNS` are defined as constants, for example `NUM_ROWS = 10` and `NUM_COLUMNS = 8`.

The value '*' is assigned to the four cells in the corners. For each remaining cell, a pseudo-random value in the range `[0, 100]` is generated:

- for every cell in the upper border, if the pseudo-random value is lower than 90, the cell is initialized with '*', otherwise with 'n'.
- for every cell in the right border, if the pseudo-random value is lower than 90, the cell is initialized with '*', otherwise with 'e'.
- for every cell in the lower border, if the pseudo-random value is lower than 90, the cell is initialized with '*', otherwise with 's'.
- for every cell in the left border, if the pseudo-random value is lower than 90, the cell is initialized with '*', otherwise with 'w'.
- for each other cell, if the pseudo-random value is lower than 60, the cell is initialized with ' ', otherwise with '*'.

The sequence of pseudo-random numbers is obtained by means of the following linear congruential generator: `value_new = (value_old * 18) % 101`, where % indicates the modulo operation.

Add the following functionalities to the project created in the previous exercise:

- 1) In the main function, start timer 0, without generating interrupts and stopping the timer.
- 2) When the user presses the button `Key2`, we read the current value of timer 0. This is a truly random number and we use it as a seed to compute the first pseudo-random number. Then, each subsequent pseudo-random number is obtained using the previous pseudo-random number.

Example: if timer 0 is 300 and `NUM_COLUMNS = 8`, then 6 pseudo-random numbers are needed for generating the first row of the matrix (because the corners are fixed). The sequence of pseudo-random numbers is:

- `value1 = (300 * 18) % 101 = 47`
- `value2 = (47 * 18) % 101 = 38`
- `value3 = (38 * 18) % 101 = 78`
- `value4 = (78 * 18) % 101 = 91`
- `value5 = (91 * 18) % 101 = 22`
- `value6 = (22 * 18) % 101 = 93`

The first row of the matrix is `****n*n*`

- 3) Finally, call the `mazeSolver` subroutine.

Steps 2 and 3 have to be implemented inside the handler of button `Key2`.